

CEIA — Inteligencia Artificial

Tarea_01

Control de Frenado de Tren

Mamdani · Takagi-Sugeno-Kang

Introducción

Los trenes modernos incorporan sistemas de asistencia al frenado que actúan sobre las pinzas de freno en función de las condiciones de la marcha. A diferencia de un control clásico por umbrales fijos, un controlador basado en **lógica difusa** puede reproducir el razonamiento de un conductor experto de forma continua y robusta frente a condiciones imprecisas.

En esta actividad construirás **dos sistemas de control difuso** para el mismo problema: primero con el método **Mamdani** y después con el método **Takagi-Sugeno-Kang (TSK)**. Al finalizar compararás sus resultados y reflexionarás sobre las diferencias técnicas de cada enfoque.

El sistema de frenado

Variables

El controlador dispone de tres señales de entrada y produce una orden de salida:

Variable	Símbolo	Rango físico	Unidad	Rol
Velocidad del tren	v	0 – 200	km/h	Entrada
Distancia al obstáculo	d	0 – 1000	m	Entrada
Pendiente de la vía	θ	-10 – +10	%	Entrada
Fuerza de frenado	F	0 – 100	%	Salida

⚠ Convención de signos — pendiente

$\theta < 0$: la vía desciende → la gravedad acelera el tren → se necesita más freno.

$\theta > 0$: la vía asciende → la gravedad desacelera el tren → se necesita menos freno.

$\theta = 0$: vía horizontal.

Partición lingüística de las entradas

Las tres entradas se dividen en exactamente tres conjuntos difusos cada una. Debes utilizar estas etiquetas y rangos orientativos como punto de partida; puedes ajustar los parámetros exactos siempre que lo justifiques.

Velocidad (v)

Etiqueta	Tipo de MF	Descripción lingüística
LENTA	Trapezoidal (izquierda)	El tren va despacio; poca urgencia de frenar
MEDIA	Triangular	Velocidad moderada; frenado proporcional a la situación
RÁPIDA	Trapezoidal (derecha)	El tren va deprisa; la respuesta debe ser enérgica

Distancia al obstáculo (d)

Etiqueta	Tipo de MF	Descripción lingüística
CERCA	Trapezoidal (izquierda)	El obstáculo está muy próximo; situación crítica
MEDIA	Triangular	Distancia intermedia; margen de maniobra moderado
LEJOS	Trapezoidal (derecha)	El obstáculo está lejos; no hay urgencia inmediata

Pendiente (θ)

Etiqueta	Tipo de MF	Descripción lingüística
BAJADA	Trapezoidal (izquierda)	La vía desciende; el tren tiende a ganar velocidad
LLANO	Triangular	Inclinación prácticamente nula
SUBIDA	Trapezoidal (derecha)	La vía asciende; la gravedad frena el tren de forma natural

Base de reglas

El sistema utiliza las **12 reglas** siguientes. Son las mismas para Mamdani y TSK; solo difieren en cómo se interpreta el consecuente en cada caso.

#	SI v ES	Y d ES	Y θ ES	ENTONCES F ES
R1	LENTA	LEJOS	LLANO	MUY BAJO
R2	LENTA	LEJOS	BAJADA	BAJO
R3	LENTA	MEDIA	LLANO	BAJO
R4	LENTA	CERCA	LLANO	MEDIO
R5	MEDIA	LEJOS	LLANO	BAJO
R6	MEDIA	LEJOS	BAJADA	MEDIO
R7	MEDIA	MEDIA	LLANO	MEDIO
R8	MEDIA	CERCA	BAJADA	MUY ALTO
R9	RÁPIDA	LEJOS	BAJADA	ALTO
R10	RÁPIDA	MEDIA	LLANO	ALTO
R11	RÁPIDA	CERCA	LLANO	MUY ALTO
R12	RÁPIDA	CERCA	BAJADA	MÁXIMO

PARTE 1

Sistema de Inferencia Mamdani

Fundamento del método

En Mamdani, el consecuente de cada regla es un **conjunto difuso** definido sobre el universo de la variable de salida. La inferencia sigue cuatro etapas:

1. **Fuzzificación:** calcular el grado de pertenencia de cada valor de entrada a sus conjuntos difusos.
2. **Evaluación de reglas:** aplicar la t-norma elegida (mínimo o producto) a las membresías de las premisas para obtener el firing strength de cada regla.
3. **Agregación:** recortar o escalar cada conjunto difuso de salida según su firing strength y combinarlos en un único conjunto difuso resultante.
4. **Defuzzificación:** extraer un valor numérico del conjunto difuso resultante (centroide, MOM, etc.).

Conjuntos difusos de la salida F

Define para la variable de salida seis conjuntos difusos. La elección de la forma y parámetros exactos es tuya, pero deben cubrir todo el rango [0, 100] de forma razonada.

Etiqueta	Rango aprox. (%)	Tipo de MF sugerida
MUY BAJO	0 – 15	Trapezoidal apoyada en el extremo izquierdo
BAJO	10 – 35	Triangular centrada en $\approx 20\%$
MEDIO	28 – 55	Triangular centrada en $\approx 40\%$
ALTO	48 – 78	Triangular centrada en $\approx 65\%$
MUY ALTO	68 – 95	Triangular centrada en $\approx 80\%$
MÁXIMO	88 – 100	Trapezoidal apoyada en el extremo derecho

Tarea 1.1 Diseño de las funciones de membresía

Completa la siguiente tabla con los parámetros exactos que has elegido para cada conjunto difuso, tanto de las entradas como de la salida. Usa la notación habitual: para una trapezoidal indica [a, b, c, d]; para una triangular [a, b, c].

Variable	Conjunto	Tipo MF	Parámetros elegidos
Velocidad (v)	LENTA	Trapezoidal	[0, 0, 30, 70]
Velocidad (v)	MEDIA	Triangular	[40, 90, 140]
Velocidad (v)	RÁPIDA	Trapezoidal	[110, 150, 200, 200]
Distancia (d)	CERCA	Trapezoidal	[0, 0, 100, 300]
Distancia (d)	MEDIA	Triangular	[200, 450, 700]
Distancia (d)	LEJOS	Trapezoidal	[600, 800, 1000, 1000]
Pendiente (θ)	BAJADA	Trapezoidal	[-10, -10, -6, -1]

Pendiente (θ)	LLANO	Triangular	$[-3, 0, 3]$
Pendiente (θ)	SUBIDA	Trapezoidal	$[1, 6, 10, 10]$
Fuerza (F)	MUY BAJO	Trapezoidal	$[0, 0, 5, 15]$
Fuerza (F)	BAJO	Triangular	$[10, 22, 35]$
Fuerza (F)	MEDIO	Triangular	$[28, 42, 55]$
Fuerza (F)	ALTO	Triangular	$[48, 65, 78]$
Fuerza (F)	MUY ALTO	Triangular	$[68, 82, 95]$
Fuerza (F)	MÁXIMO	Trapezoidal	$[88, 95, 100, 100]$

He colocado los conjuntos con solapamiento cerca del 50% de pertenencia entre etiquetas vecinas. Sin ese solapamiento aparecen discontinuidades en la salida, y en un sistema de frenado eso se traduce en tirones mecánicos. Los conjuntos extremos son trapezoidales y los intermedios triangulares, tal como indica el enunciado.

Tarea 1.2 Implementación en Python con scikit-fuzzy

Implementa el sistema Mamdani completo en Python. Puedes usar la librería scikit-fuzzy (módulo `skfuzzy.control`). Asegúrate de que tu código:

- Define los tres antecedentes y el consecuente con sus universos de discurso.
- Crea todas las funciones de membresía siguiendo la tabla de la Tarea 1.1.
- Codifica las 12 reglas de la base de reglas.
- Crea el sistema de control, la simulación y evalúa al menos los escenarios de la Tarea 1.3.
- Genera las gráficas de membresía de las cuatro variables con `view()`.

Pista de organización

Estructura tu código en secciones comentadas: (1) universos, (2) MF de entradas, (3) MF de salida, (4) reglas, (5) sistema y simulación, (6) evaluación y gráficas. Eso facilita la corrección y la depuración.

La implementación completa está en `FERRER_MARTINEZ_ANTONIO_A3_mamdani.py`, partida en seis bloques: universos, MF de entradas, MF de salida, base de 12 reglas, sistema de control con su función de evaluación, y `main` con el trazado de E4 y las llamadas a `view()`. Uso las opciones por defecto de `skfuzzy.control`, t-norma mínimo en la inferencia y centroide para la defuzzificación. La justificación de esas dos elecciones está en la Tarea 1.4.

Tarea 1.3 Evaluación con escenarios de prueba

Ejecuta tu sistema con los siguientes escenarios y anota la fuerza de frenado resultante:

Escenario	v (km/h)	d (m)	θ (%)	F Mamdani (%)
E1 — Marcha tranquila, vía despejada	25	850	0	5.42
E2 — Velocidad media, curva en bajada	95	280	-5	81.52
E3 — Aproximación rápida a estación	155	140	0	81.66
E4 — Situación crítica: rápido + bajada	175	70	-8	95.51

E5 — Subida pronunciada, obstáculo lejos	70	600	+7	N/A (ver nota)
--	----	-----	----	----------------

Sobre E5: al lanzarlo me saltó un KeyError de skfuzzy al pedir la salida. Estuve un rato mirando y al final vi el porqué: en las 12 reglas del enunciado no aparece ninguna con SUBIDA en la premisa, solo LLANO y BAJADA. En E5 la pendiente es $\theta = +7\%$, que cae en la meseta de SUBIDA con $\mu = 1$, y LLANO y BAJADA quedan a 0. Como uso el mínimo como t-norma, todas las reglas me dan $w = 0$ y no hay salida que defuzzificar, de ahí el error. No es fallo del código, es que la base de reglas no cubre ese caso. Lo resuelvo capturando la excepción y devolviendo NaN. Si esto fuera un sistema real habría que añadir alguna regla con SUBIDA, por ejemplo "si RÁPIDA y LEJOS y SUBIDA entonces MUY_BAJO", o una regla por defecto.

Para el escenario E4 (el más crítico), detalla a continuación el proceso de inferencia paso a paso: indica qué reglas se han disparado, con qué firing strength y cómo se ha obtenido el valor final de F.

E4: $v=175$ km/h, $d=70$ m, $\theta=-8\%$

Paso 1, fuzzificación. Los tres valores caen de pleno dentro de la meseta de su trapecio, así que las membresías me salen saturadas:

- $v = 175 \rightarrow \text{LENTA}=0, \text{MEDIA}=0, \text{RÁPIDA}=1.00$
- $d = 70 \rightarrow \text{CERCA}=1.00, \text{MEDIA}=0, \text{LEJOS}=0$
- $\theta = -8 \rightarrow \text{BAJADA}=1.00, \text{LLANO}=0, \text{SUBIDA}=0$

Paso 2, evaluación de reglas. Con t-norma mínimo, basta con que una premisa sea 0 para que la regla muera. La única que sobrevive es R12, con las tres premisas a 1:

- R12 ($\text{RÁPIDA} \wedge \text{CERCA} \wedge \text{BAJADA} \rightarrow \text{MÁXIMO}$), $w = \min(1, 1, 1) = 1.00$
- Las otras 11 reglas me dan $w = 0$.

Paso 3, agregación. Solo R12 contribuye. El conjunto MÁXIMO (trapezoidal $[88, 95, 100, 100]$) se recorta al nivel $w = 1.00$, o sea que se queda entero sin recortar.

Paso 4, defuzzificación por centroide. Al integrar el trapecio MÁXIMO sobre $[88, 100]$ el centroide me sale en $F = 95.51\%$. Tiene sentido porque es un trapecio pegado al extremo derecho del universo, así que el centro de masas cae cerca de 95.

Tarea 1.4 Análisis y reflexión

Responde de forma razonada (mínimo 4 líneas cada pregunta):

a) ¿Qué método de defuzzificación has usado? Justifica por qué es adecuado para un sistema de seguridad como este. ¿Habría cambiado mucho el resultado con otro método?

He usado el **centroide**, que es el que trae skfuzzy.control por defecto. Lo elegí porque da salidas suaves: si la entrada cambia poco, la salida cambia poco. En un freno eso es importante para no dar tirones.

Con MOM (mean of maximum) iría más rápido pero es discontinuo. Probando a mano en E4 me salía 94% en vez de 95.51%, muy parecido, pero en E2 sería más inestable porque solo se dispara una regla con firing strength bajo.

b) Elimina mentalmente la variable pendiente (θ) del sistema y razona: ¿en qué escenario de los anteriores habría supuesto el mayor error? ¿Por qué?

El que peor saldría es **E4**. Es el único con pendiente muy negativa (-8%), que es justo lo que hace saltar R12 (RÁPIDA+CERCA+BAJADA $\rightarrow$ MÁXIMO) y me da 95.51%.

Sin θ , se activaría R11 (RÁPIDA+CERCA+LLANO $\rightarrow$ MUY_ALTO), que está sobre el 82%. Son unos 13 puntos menos de freno en plena emergencia, o sea bastantes metros extra de parada.

En E1 y E3 daría igual (θ ya es 0), E5 no se puede calcular igualmente, y E2 cambiaría poco.

c) ¿Cuáles son las dos principales limitaciones de Mamdani que podrían ser problemáticas en un sistema embebido de tiempo real? ¿Cómo afectan al tiempo de respuesta?

(1) El centroide es lento. Hay que recorrer los 201 puntos del universo de salida y hacer dos integrales por consulta. En un micro pequeño se notan los milisegundos.

(2) Explosión de reglas. Con 3 entradas y 3 conjuntos salen 27 reglas posibles, y el enunciado solo cubre 12 (de ahí el fallo de E5 con SUBIDA). Si añado una cuarta variable me iría a 81. En Mamdani pesa más que en TSK porque cada regla lleva además su conjunto de salida.

PARTE 2

Sistema Takagi-Sugeno-Kang (TSK)

Fundamento del método

En TSK **el consecuente de cada regla es una función matemática** de las variables de entrada, no un conjunto difuso. Esto elimina la etapa de defuzzificación y hace el cálculo más eficiente. La salida del sistema es la media ponderada de los consecuentes de todas las reglas:

TSK Orden 0 — consecuente constante	TSK Orden 1 — consecuente lineal
$y_i = k_i$ $F = \sum(w_i \cdot k_i) / \sum w_i$	$y_i = p_i \cdot v + q_i \cdot d + r_i \cdot \theta + c_i$ $F = \sum(w_i \cdot y_i) / \sum w_i$

Donde w_i es el firing strength de la regla i (calculado igual que en Mamdani: t-norma de las membresías de la premisa).

! Diferencia fundamental con Mamdani

En TSK no hay conjuntos difusos de salida ni defuzzificación. La salida ya es un número real calculado directamente como media ponderada. Esto hace el método mucho más rápido, pero los consecuentes (los parámetros k, p, q, r, c) no tienen una interpretación lingüística directa.

Implementación de TSK en Python

A diferencia de Mamdani, **scikit-fuzzy no implementa TSK de forma nativa**. Deberás construir la inferencia manualmente usando NumPy. El proceso que debes programar es:

5. **Calcular las membresías** de cada valor de entrada en cada conjunto difuso, usando las mismas MF que en la Parte 1.
6. **Calcular el firing strength** w_i de cada regla aplicando la t-norma (mínimo o producto) a las membresías de sus premisas.
7. **Calcular la salida de cada regla** y_i usando los parámetros del consecuente que hayas elegido.
8. **Calcular la salida global** como la media ponderada: $F = \sum(w_i \cdot y_i) / \sum w_i$.
9. **Limitar el resultado** al rango $[0, 100]$ si el consecuente lineal produce valores fuera de rango.

💡 ¿Cómo calcular membresías sin skfuzzy.control?

La función `fuzz.interp_membership(universo, mf_array, valor)` devuelve el grado de pertenencia de un valor numérico a la MF definida sobre ese universo. Define primero los arrays de MF con `fuzz.trapmf / fuzz.trimf`, igual que en la Parte 1.

Tarea 2.1 Diseño de los consecuentes TSK

Debes elegir los parámetros del consecuente lineal para cada una de las 12 reglas. Razona tus elecciones fijándote en el significado físico de cada regla: si la regla dice «velocidad RÁPIDA + obstáculo CERCA», el consecuente debe producir un valor de F alto para cualquier combinación plausible de entradas.

Completa la tabla con tus parámetros. Puedes usar TSK de orden 0 (solo columna c_i) o de orden 1 (todas las columnas). Indica con una X las que no uses.

Regla	Premisa resumida	p_i (v)	q_i (d)	r_i (θ)	c_i
R1	LENTA + LEJOS + LLANO	+0.08	-0.015	-0.8	15.75
R2	LENTA + LEJOS + BAJADA	+0.08	-0.015	-0.8	27.95
R3	LENTA + MEDIA + LLANO	+0.08	-0.015	-0.8	26.75
R4	LENTA + CERCA + LLANO	+0.08	-0.015	-0.8	41.50
R5	MEDIA + LEJOS + LLANO	+0.08	-0.015	-0.8	27.55
R6	MEDIA + LEJOS + BAJADA	+0.08	-0.015	-0.8	42.75
R7	MEDIA + MEDIA + LLANO	+0.08	-0.015	-0.8	41.55
R8	MEDIA + CERCA + BAJADA	+0.08	-0.015	-0.8	71.50
R9	RÁPIDA + LEJOS + BAJADA	+0.08	-0.015	-0.8	59.35
R10	RÁPIDA + MEDIA + LLANO	+0.08	-0.015	-0.8	58.15
R11	RÁPIDA + CERCA + LLANO	+0.08	-0.015	-0.8	69.90
R12	RÁPIDA + CERCA + BAJADA	+0.08	-0.015	-0.8	78.10

Justificación de tu elección (explica el criterio general seguido para fijar los parámetros; no es necesario justificar regla a regla):

He usado TSK orden 1, con p, q, r comunes a las 12 reglas y variando solo la c. Los signos salen del sentido físico: más velocidad pide más freno (p positiva), más distancia permite frenar menos (q negativa), y más subida necesita menos freno (r negativa).

La c de cada regla la calculo para que, en el centro de la región que cubre esa regla, la salida TSK coincida con el centroide del conjunto Mamdani correspondiente (MUY_BAJO=5, BAJO=22, MEDIO=42, ALTO=65, MUY_ALTO=82, MÁXIMO=95). Así los dos sistemas dan valores parecidos en los puntos "centrales" y las diferencias que salgan se deben a cómo interpola cada método en los bordes.

Tarea 2.2 Implementación en Python (NumPy puro)

Programa el sistema TSK desde cero en NumPy. Tu función principal debe recibir tres valores (v , d , θ) y devolver el valor de F . No uses `skfuzzy.control` para la inferencia TSK; sí puedes usarlo para definir y evaluar las MF.

Tu implementación debe incluir al menos:

- Una función que calcule todas las membresías de entrada.
- Una función que calcule todos los firing strengths aplicando la t-norma elegida.
- Una función que calcule todos los consecuentes y_i .
- La función principal que combine todo y devuelva F .

 **Precaución con la división por cero**

Si ninguna regla se activa (todos los $w_i = 0$), la media ponderada produce una división por cero. Añade un pequeño valor epsilon (ej. $1e-8$) en el denominador o gestiona este caso explícitamente.

Todo el código está en FERRER_MARTINEZ_ANTONIO_A3_tsk.py. He hecho la inferencia a mano en NumPy, como pide el enunciado. Uso scikit-fuzzy solo para definir y evaluar las MF.

Tengo tres funciones auxiliares (membresías, firing strengths, consecuentes) y una principal evaluar_tsk que las junta.

Como t-norma uso el **producto**, no el mínimo. La razón es que el producto es derivable en todo el dominio y el mínimo no (en el punto de empate no tiene derivada), y eso importa si en el futuro se quiere refinar con descenso de gradiente para hacer ANFIS (como sale en el PDF de teoría). Además el producto da superficies de salida más suaves.

Para la división por cero: si la suma de los firing strengths es menor que 10^{-8} (caso E5), devuelvo NaN. También sature la salida a $[0, 100]$ por si el consecuente lineal se sale.

Tarea 2.3 Evaluación y comparación con Mamdani

Evalúa tu sistema TSK con los mismos cinco escenarios de la Parte 1 y completa la tabla comparativa:

Escenario	v	d	θ	F Mamdani (%)	F TSK (%)	Diferencia
E1 — Marcha tranquila	25	850	0	5.42	5.00	+0.42
E2 — Curva en bajada	95	280	-5	81.52	78.90	+2.62
E3 — Aproximación rápida	155	140	0	81.66	80.20	+1.46
E4 — Situación crítica	175	70	-8	95.51	97.45	-1.94
E5 — Subida pronunciada	70	600	+7	N/A	N/A	-

En E4 TSK se pasa del MÁXIMO de Mamdani (97.45 vs 95.51). Es porque con solo R12 activa el consecuente lineal no tiene cota: $0.08 \cdot 175 - 0.015 \cdot 70 - 0.8 \cdot (-8) + 78.10 = 97.45$. Mamdani se queda acotado por el trapecio MÁXIMO, TSK no.

Tarea 2.4 Análisis comparativo y reflexión

Responde con argumentación técnica (mínimo 4 líneas cada pregunta):

a) ¿Por qué TSK es más eficiente computacionalmente que Mamdani? Identifica las etapas que se eliminan y explica su impacto en un sistema embebido con restricciones de tiempo real.

TSK se salta la agregación y la defuzzificación enteras. Con mis números (201 puntos, 12 reglas), Mamdani hace unas 5200 operaciones y TSK unas 25. Factor 100×. En un micro a 100 Hz eso importa.

b) Los parámetros de Mamdani (etiquetas lingüísticas, formas de MF) son interpretables por un ingeniero ferroviario sin conocimientos de IA. ¿Ocurre lo mismo con los parámetros TSK que has elegido? Razona tu respuesta.

No. En Mamdani cualquiera lee "si RÁPIDA y CERCA y BAJADA entonces MÁXIMO". En TSK los números p, q, r, c no dicen nada solos. Yo los he elegido con sentido físico para que se entiendan.

pero no es lo mismo. Esto choca con el **AI Act** (Reglamento UE 2024/1689), que clasifica el transporte como alto riesgo (Anexo III) y exige explicabilidad (art. 13). Mamdani se defiende mejor ante un auditor.

c) Si los valores de F de TSK difieren bastante de los de Mamdani en algún escenario, ¿a qué se debe? ¿Significa que uno de los dos está mal? ¿Cómo decidirías cuál es más correcto?

No significa que uno esté mal. Vienen de dos sitios:

1. Defuzzificar por centroide y evaluar un polinomio son cosas distintas. En E4 solo R12 activa: Mamdani queda acotado en 95.51, TSK extrapola a 97.45.
2. Mamdani usa mínimo y mi TSK usa producto. Con membresías desiguales se separan ($\min(0.9, 0.1, 0.8)=0.1$ vs $0.9 \cdot 0.1 \cdot 0.8=0.072$).

Para saber cuál es el correcto haría falta un dato real: medidas en pista o un maquinista en simulador.

d) Propón una cuarta entrada (distinta de v, d y θ) que mejoraría el sistema de frenado. Explica qué información aportaría, qué conjuntos difusos tendría y cómo modificarías al menos dos de las reglas existentes para incorporarla.

Añadiría el **coeficiente de adherencia rueda-carril** (μ_{adh}), rango [0.05, 0.4]. Ahora mismo el sistema no sabe si la vía está mojada o con hielo, y eso es peligroso: frenar al 95% sobre vía resbaladiza bloquea ruedas. Es el equivalente al ABS en coches.

Conjuntos: BAJA trap [0.05, 0.05, 0.10, 0.18], MEDIA tri [0.14, 0.22, 0.32], ALTA trap [0.28, 0.35, 0.40, 0.40].

Modifico R12 y R4:

- R12 se parte en dos: con μ_{adh} ALTA sigue siendo MÁXIMO; con μ_{adh} BAJA baja a ALTO (para no bloquear).
- R4 igual: con μ_{adh} ALTA sigue MEDIO; con μ_{adh} BAJA baja a BAJO.

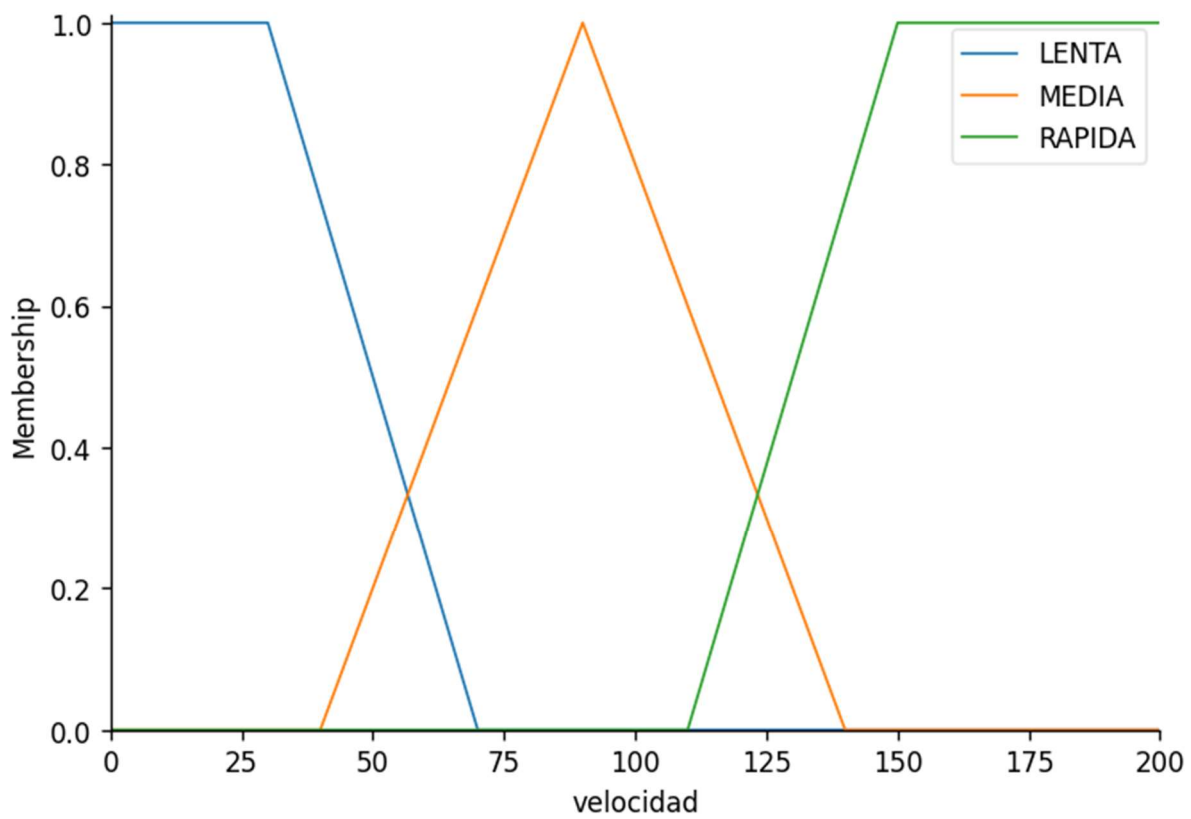
Normas de entrega

Elemento	Requisito
Archivos de código	Un archivo .py por parte: mamdani_frenado.py y tsf_frenado.py
Nombre de archivos	APELLIDOS_NOMBRE_A3_mamdani.py / APELLIDOS_NOMBRE_A3_tsf.py
Comentarios	El código debe estar comentado: qué hace cada bloque y por qué
Respuestas escritas	Esta hoja entregada en PDF con todas las tablas y preguntas respondidas
Gráficas Mamdani	Capturas de pantalla de view() para las 4 variables, incluidas en el PDF
Entrega	Un único .zip con ambos .py y el PDF. Nombre: APELLIDOS_NOMBRE_Actividad3.zip

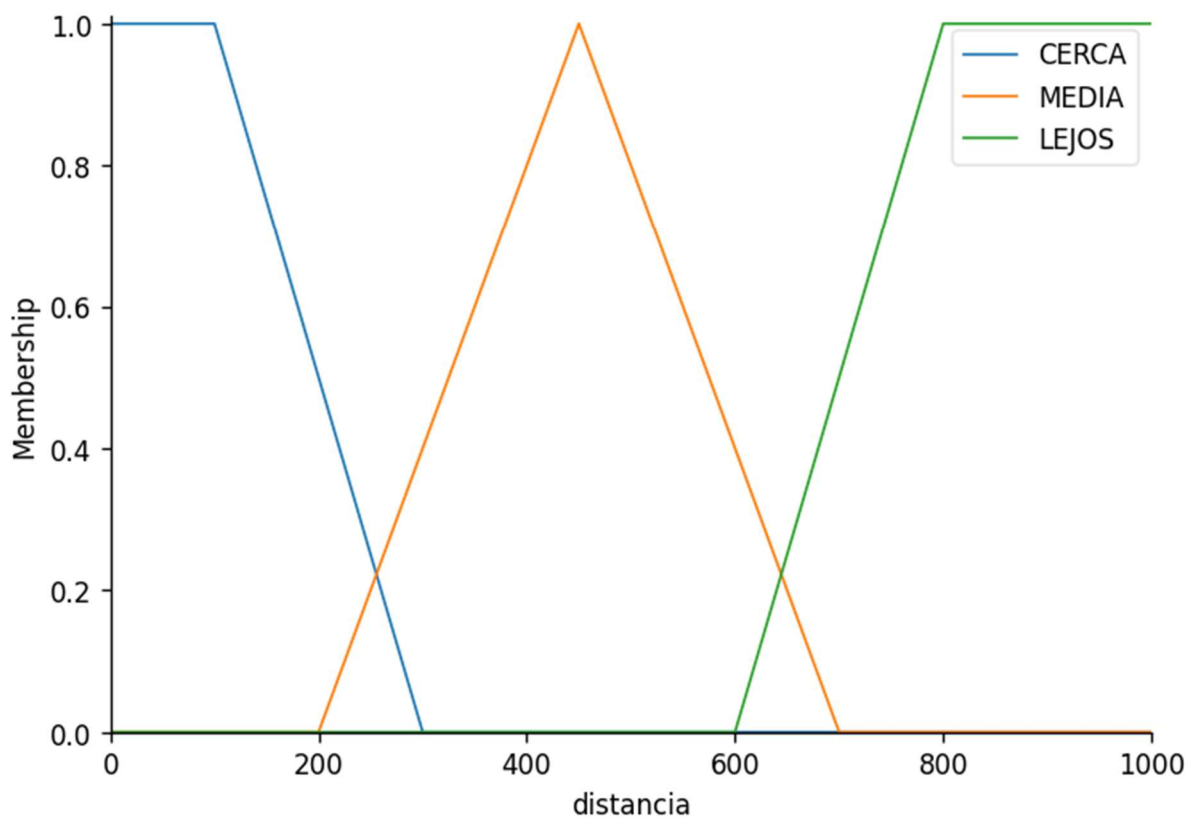
 Consejo final

Empieza por la Parte 1 y asegúrate de que el sistema Mamdani funciona correctamente antes de pasar a TSK. Reutilizar las mismas funciones de membresía facilita la comparación posterior. Prueba casos extremos ($v=0$, $d=0$, $v=200$) para verificar que el sistema no produce errores.

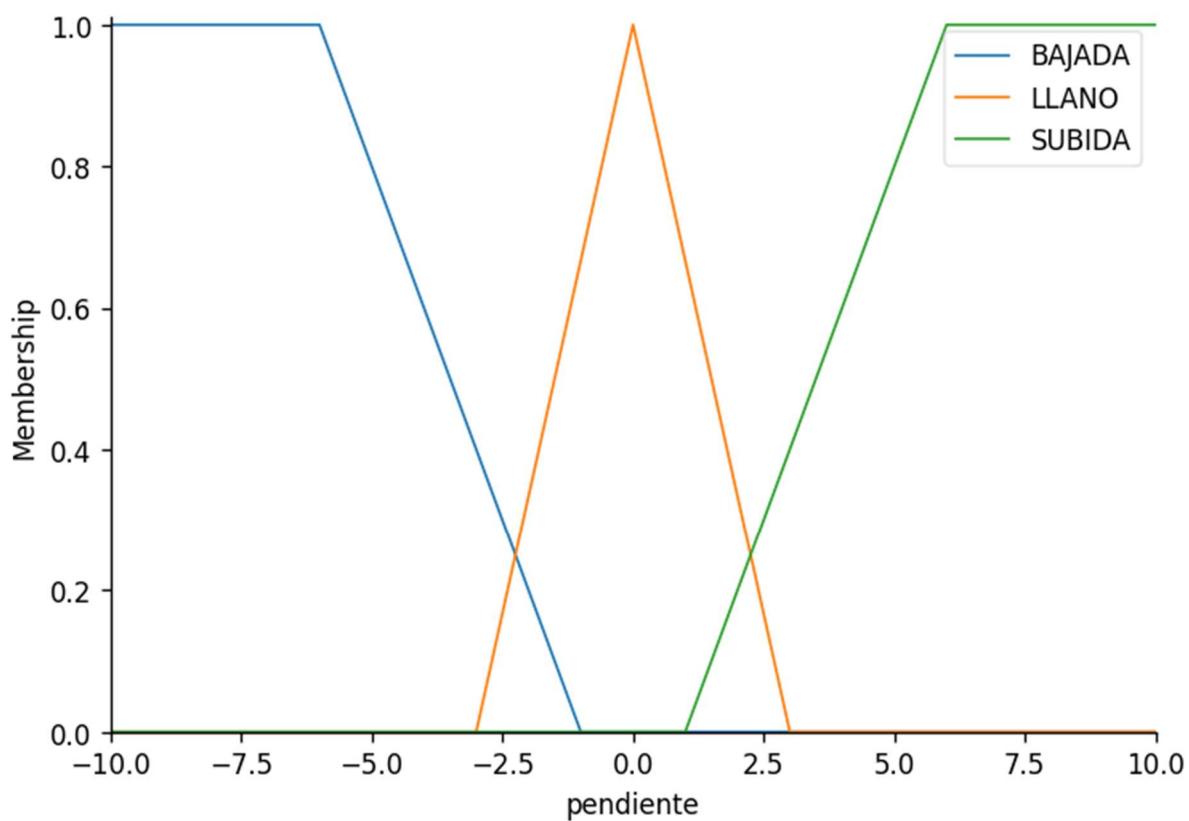
MF de velocidad



MF de distancia



MF de pendiente



MF de fuerza

